



University of Trento

Department of Information Engineering and Computer Science
Artificial Intelligence Systems

Board Game Analysis: Score Tracking on the Hegemony Board

Authors:

Federico Berra
Davide Verzotto

July 2026

Contents

1	Introduction	2
2	System Architecture and Library Choices	2
3	Board Rectification and Frame Alignment	2
3.1	Rectified Reference and Track Mask	2
3.2	Feature-Based Localization	3
3.3	Guarded ECC Refinement	4
4	Score Track Model	5
5	Score Disc Detection	6
5.1	Reference Difference and Blob Filtering	6
5.2	Colour Classification and Cell Assignment	6
6	Game Tracking and Results	7
6.1	Temporal Filtering	7
6.2	Evaluation	8
7	Edge Operators and Preprocessing	9
8	Conclusions and Future Work	10

1 Introduction

This report presents the analysis of Hegemony, an asymmetric board game that simulates the socioeconomic policies of an entire nation through cards. In particular, the first part requires the automatic localization and rectification of the game board from photographs taken during a real match, while the second part focuses on reading the scores of the four playing classes (Working, Capitalist, Middle and State). Our system processes a sequence of frames captured during a real game. The images contain different camera viewpoints, reflections and frequent hand occlusions. The system combines robust board alignment, colour-based detection of the score discs, and temporal filtering of the readings, and is evaluated against a manually annotated gold standard, reaching 92% exact readings and 94% within one cell on the benchmark game.

2 System Architecture and Library Choices

The implementation relies on three standard Python libraries:

- **OpenCV (cv2):** ORB features, matching (BFMatcher, GMS), robust estimation (RANSAC, ECC), image processing.
- **NumPy:** array operations, statistics, and reproducibility (seeds).
- **Matplotlib:** visualization and score timeline plots.

3 Board Rectification and Frame Alignment

Before reading the scores, every frame is aligned with the rectified reference image. This section explains how the reference image is created, how each frame is aligned using feature matching, and how a final refinement step further improves the alignment.

3.1 Rectified Reference and Track Mask

The first frame of each session, captured before any score is recorded, is used to create a canonical reference. It is not a photograph of an empty board: the game has already been set up, but the score track, the only region from which scores are read, is still empty. Therefore, the central area of the board is identical to the reference only at the start of the game and changes as players move their pieces. For this reason, the pipeline focuses exclusively

on the score track by applying the ring mask described below. Four corner points are manually selected using a simple OpenCV point-and-click tool, and a perspective transform is computed to warp the original photograph into a fixed-size top-down view. This is the only manual operation; all subsequent steps are fully automatic (Section 3.2).

This rectified image defines the common coordinate system used throughout the pipeline. Every subsequent frame is projected into this reference and the same image is later used as a background for scoring detection (Section 5).

Since only the outer score track is needed, a binary ring mask is created to exclude the central play area. The mask is made up of four rectangular strips along the upper, lower, left, and right borders. Their thickness is chosen to fully cover the score track while excluding most of the central area.

The same mask is used throughout the pipeline to limit correlation, ECC refinement, and foreground extraction to the score track only.

3.2 Feature-Based Localization

Each frame is usually captured from a slightly different viewpoint and under different lighting conditions. Therefore, it must first be aligned with the reference image.

We first compute an initial alignment by extracting ORB (Oriented FAST and Rotated BRIEF) keypoints and descriptors from both the reference and the input frame. Matches are computed with a brute-force Hamming-distance matcher, then filtered to keep only the geometrically consistent ones. When available, we use GMS (Grid-based Motion Statistics) filtering; otherwise we fall back on Lowe’s ratio test with a 0.75 threshold.

The remaining matches are used to estimate a partial affine transform with RANSAC, which makes the estimate robust to the leftover outliers. This transformation only allows translation, rotation and uniform scaling. This is enough in our case because the board is almost fronto-parallel, while the remaining perspective distortion is corrected during ECC refinement. We then embed it into a 3×3 homography, so that the whole frame can be warped into the reference coordinate system with a single `warpPerspective` call. At the same time, we build a validity mask that marks which pixels come from the input image and which are just extrapolated border regions. Before any further processing, we slightly erode this mask to remove the unreliable pixels near the image edges.

3.3 Guarded ECC Refinement

Although the similarity registration is generally accurate, errors of just a few pixels are enough to leave residual traces of the printed track numbers in the image difference (Section 5). To fix this, we refine the alignment with the Enhanced Correlation Coefficient (ECC) algorithm, which estimates a full homography maximizing the correlation between the aligned frame and the reference. Optimization is restricted to the score-track mask, reducing the influence of occlusions, tokens, or hands elsewhere in the image.

ECC does not always help: under difficult lighting, or when the initial registration is already very accurate, it can even worsen the result. We therefore attempt refinement at two Gaussian smoothing levels (21 and 5 pixels, coarse to fine) and, after each step, score the alignment with the Pearson correlation over the valid track pixels. We keep the new alignment only if its correlation score is higher than the current one. Otherwise, the previous alignment is kept (Listing 1).

```
def align_to_reference(frame):
    ...
    for g in (21, 5):
        cand, cand_valid = refine_with_ecc(best_out,
            fixed_emptyBoard, mask=ring_mask, gauss=g,
            valid_in=best_valid)
        s = alignment_score(cand, fixed_emptyBoard,
            valid=cand_valid)
        if s > best_s:
            best_out, best_valid, best_s = cand,
                cand_valid, s

    if best_s <= 0.10:
        return None, None
    return best_out, cv2.erode(best_valid, np.ones((7,
        7), np.uint8))
```

Listing 1: Guarded ECC refinement

If the final correlation score remains below 0.10 after both refinement attempts, the frame is discarded, as it likely corresponds to a failed registration, severe occlusion, or excessive motion blur; on the benchmark game no frame was rejected. Figure 1 shows a frame before and after alignment.



Figure 1: Original frame and frame aligned to the rectified reference

4 Score Track Model

Once frames are aligned, we need the image position of each 0–99 score cell to map a detected disc to its score. Since the cells are evenly spaced along the four board edges, we compute their centres analytically instead of annotating them by hand: for each side we measure the two endpoint cells on the rectified reference and interpolate the rest uniformly between them. Figure 2 overlays the resulting cell centres on the rectified reference for verification, each label should fall on its printed cell.



Figure 2: Verification overlay of the track model on the rectified reference

5 Score Disc Detection

With the frame aligned to the reference, the two images match pixel by pixel except where something has been added. The difference between the two images highlights the objects placed on the score track, mainly the score discs and occasionally the players' hands, plus faint residues of the printed numbers where the alignment is not pixel-exact. Figure 3 shows the result on a single frame.

5.1 Reference Difference and Blob Filtering

We threshold the difference between the aligned frame and the reference, taking the maximum across the three channels and keeping only pixels that changed by more than `DIFF_THRESHOLD`. The surviving blobs are then filtered by area, to remove both tiny noise regions and oversized ones, and by bounding-box side and aspect ratio, which rejects the players' hands and arms that are far larger than a disc (Listing 2).

```
DIFF_THRESHOLD = 60    # threshold on the difference
MIN_TOKEN_AREA = 60    # minimum blob area (px^2)
MAX_TOKEN_AREA = 2000  # maximum blob area
MAX_TOKEN_SIZE = 35    # max box side (rejects hands)
```

Listing 2: Detection parameters

5.2 Colour Classification and Cell Assignment

Each blob is classified using the median HSV value of a 15×15 patch around its centroid, which is more stable than reading a single pixel. Saturation and value first tell the grey discs apart from the coloured ones, and the colour tone then picks the class. Finally, the blob is snapped to the nearest track cell to read its score (Listing 3).

```
def classify_token_color(hsv):
    H, S, V = int(hsv[0]), int(hsv[1]), int(hsv[2])
    if S < 55 and V > 110:
        return "State"
    if S >= 55 and V >= 80:
        if (H <= 15 or H >= 150) and V > 140:
            return "Working"
        if 15 <= H <= 40:
            return "Middle"
```

```

if 90 <= H <= 135:
    return "Capitalist"
return None

```

Listing 3: HSV-based class assignment



Figure 3: Detected score discs on a single frame

6 Game Tracking and Results

We apply the single-frame reading to every frame of the game and combine the results into a per-class score timeline, with temporal filtering to absorb missed or wrong detections.

6.1 Temporal Filtering

Reading each frame independently produces a noisy per-class sequence due to occlusions and hands covering the track. To reduce these errors, we combine the readings using three consecutive steps, operating on the unfiltered list of blobs detected in each frame rather than on the single-frame estimate. When multiple blobs of the same class remain after filtering, the one closest to the previous score of that class is selected. The single-frame detection shown in Figure 3 does not have access to a previous score and therefore selects the blob with the largest area instead. A width-3 median filter is then applied to remove isolated spikes, and finally a monotonic step enforces non-decreasing

scores by computing the running maximum (Listing 4). This reflects the rules of the game, but it also means that an overestimated score cannot be corrected in later frames. A representative example is frame 019 (State predicted as 20 vs. actual 11): the disc is detected one annotated frame too early near cell 20, and the median filter cannot correct the error because the true score reaches 20 in the following frame. The largest deviations in the sequence have a different cause, which is discussed in Section 6.2.

```
def isotonic(values):
    result = []
    current_max = 0 # start of the game: score 0
    for v in values:
        if v is not None:
            current_max = max(current_max, v)
        # also for None values: carry the last value
        result.append(current_max)
    return result
```

Listing 4: Running maximum to enforce non-decreasing scores

6.2 Evaluation

The gold standard was built by manually annotating the four scores on every frame, giving a per-frame ground truth against which the pipeline is measured. Since scores can only increase, we automatically check that the annotated sequences are non-decreasing, which did catch one transcription error. We used this reference not only for the final evaluation but throughout development, iterating on the detection parameters (Listing 2) and the HSV thresholds (Listing 3) until the readings matched the annotations. For this reason, the reported results should be considered slightly optimistic, since the same game was used both for parameter tuning and for evaluation.

Game 2 was selected as the benchmark after inspecting five different recorded games: it offers the cleanest and most complete view of the score track, making it the most suitable for a reliable quantitative evaluation. On its 44 annotated frames the pipeline reaches 92% exact readings and 94% within ± 1 , with a mean absolute error of only **0.31** cells (Table 1). Capitalist is read correctly on every frame. Middle and State share the lowest exact rate (86%). However, their error patterns are different: none of Middle’s errors are near misses, since their exact and within ± 1 rates coincide, while State is incorrect on relatively few frames but by much larger amounts, leading to the highest MAE among the four classes. Its neutral grey discs are distinguished only by low saturation and are easily confused with reflections under the

board’s warm lighting; above all, when two classes are tied the players stack one disc on top of the other, the pair merges into a single blob that the size and shape filters reject, and both classes keep their previous score. At frame 054, with Working and State both on cell 80, this produces the two largest errors of the sequence. Figure 4 shows the reconstructed score evolution, which follows the true game progression smoothly and monotonically.

Class	MAE	Exact	Within ± 1	N
Working	0.23	95%	98%	44
Capitalist	0.00	100%	100%	44
Middle	0.39	86%	86%	44
State	0.61	86%	93%	44
Total	0.31	92%	94%	176

Table 1: Score reading accuracy on Game 2

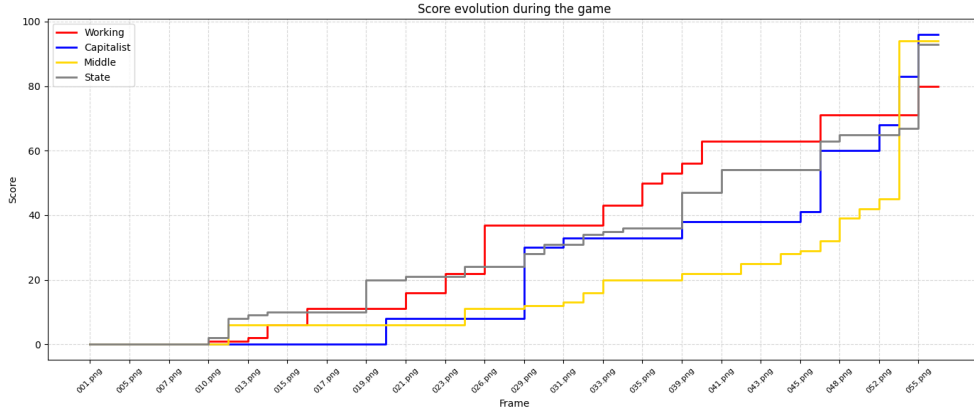


Figure 4: Score evolution of the four classes during the game

7 Edge Operators and Preprocessing

Before adopting the reference-difference method, we implemented the classic edge operators seen in class from scratch and compared them on the rectified board.

Preprocessing: We compared histogram equalization plus Gaussian smoothing against a median filter, and kept the median as input to Canny.

First-derivative operator: we implemented the 2×2 operator used in the laboratory, which differences the two rows of the neighbourhood with a single kernel rather than the two diagonal kernels of the original Roberts cross.

Second-derivative operator: For the Laplacian (3×3 , centre weight 8) we threshold the magnitude instead of locating the zero crossings.

Canny: Cleanest contours of the three, but it fires on the printed track numbers as well as on the discs (Figure 5): this is what motivated the reference-difference approach.

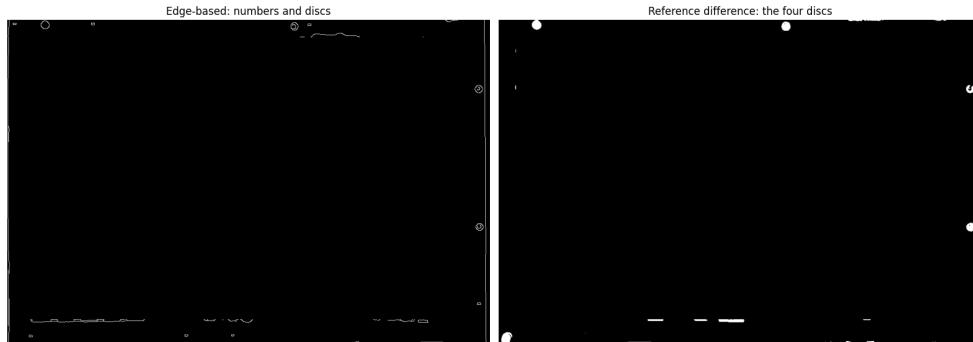


Figure 5: Edge-based mask (left) vs. reference-difference mask (right) in the track band

8 Conclusions and Future Work

The proposed pipeline successfully tracks the players' scores throughout the game by combining:

- a geometric step based on **similarity transformation** (ORB + GMS + RANSAC) refined with **homographic ECC**, providing a consistent alignment with the reference board;
- an analysis step based on the **track model** and **difference + colour classification**, improving the initial background subtraction approach.

The current approach has two main limitations. The running-maximum step enforces the game rules but is one-directional: overestimated scores cannot be corrected afterwards, so incorrect detections should be filtered before this step. Moreover, when two classes have the same score, their discs overlap and the lower one cannot be detected.

As a possible extension, the method could also be applied to other board areas, such as the politics section, improve the detection of overlapping discs using class-specific difference masks, and replace the manually tuned HSV thresholds in `classify_token_color()` with a calibrated or data-driven colour model to improve robustness under different lighting conditions.